

Package ‘concurve’

January 26, 2026

Type Package

Title Computes & Plots Compatibility (Confidence),
Surprisal, & Likelihood Distributions

Version 3.0.0

Date 2026-01-26

Maintainer Zad Rafi <zad@lesslikely.com>

Description

Computes compatibility (confidence) distributions along with their corresponding P-values, S-values, and likelihoods. The intervals can be plotted to form the distributions themselves. Functions can be compared to one another to see how much they overlap. Results can be exported to Microsoft Word, Powerpoint, and TeX documents. The package currently supports resampling methods, computing differences, generalized linear models, mixed-effects models, survival analysis, and meta-analysis. These methods are discussed by Schweder T, Hjort NL. (2016, ISBN:9781316445051) and Rafi Z, Greenland S. (2020) <doi:10.1186/s12874-020-01105-9>.

License GPL-3 | file LICENSE

URL <https://stat.lesslikely.com/concurve/>,
<https://github.com/zadrafi/concurve>

BugReports <https://github.com/zadrafi/concurve/issues>

Depends R (>= 4.0.0)

Imports methods,
bcaboot,
boot,
lme4,
dplyr,
flextable,
ggplot2,
knitr,
metafor,
officer,
parallel,
ProfileLikelihood,
scales,
colorspace,
survival,
survminer,

tibble,
tidyr,
pbmcapply

Suggests DBI, odbc, MASS, covr, roxygen2, spelling, testthat, rmarkdown,
Lock5Data, carData, bench, rms, brms, rstan, rstanarm, bayesplot,
vdiff, ggtext, daewr, svglite, data.table, nlme,
simstudy, patchwork, cowplot, reprex

VignetteBuilder knitr

ByteCompile true

Encoding UTF-8

Language en-US

LazyData true

Roxygen list(markdown = TRUE, old_usage = TRUE)

RoxygenNote 7.3.3

X-schema.org-keywords confidence, compatibility, consonance, curve,
information statistics, surprisals, interval, function, distribution, fiducial

Contents

concurve-package	3
curve_boot	4
curve_compare	5
curve_corr	7
curve_from_ratio	8
curve_from_se	9
curve_lik	10
curve_lmer	10
curve_mean	11
curve_meta	12
curve_model	14
curve_overlap	16
curve_rev	17
curve_snowflake	18
curve_snowflake_batch	19
curve_summary	21
curve_surv	22
curve_table	23
curve_wrap	24
export_for_powerbi	25
ggcurve	27
plot_compare	29
plot_multi	31
RobustMax	33
RobustMin	33

Index	34
--------------	-----------

Description

Allows one to compute compatibility (confidence) intervals for various statistical tests along with their corresponding P-values, S-values, and likelihoods. The intervals can be plotted to create consonance, surprisal, and likelihood functions allowing one to see what effect sizes are compatible with the test model at various compatibility levels rather than being limited to one interval estimate such as 95%.

Package: concurve



Logo: ""

Type: Package

Version: 3.0.0

Date: 2026-10-07

License: GLP-3

Details

Accepts most modeling functions that produce confidence intervals to construct distributions. Instructions are also provided on how to construct similar graphs for [Stata](#).

See the following articles::

- [Comparison to Bayesian Posterior Distributions](#)
- [The Bootstrap and Consonance Functions](#)
- [Background Literature](#)
- [Customizing Plots](#)
- [Examples in R](#)
- [Logistic Regression in R](#)
- [Profile Likelihoods](#)
- [Meta-Analysis Examples](#)
- [Using Stata](#)
- [Survival Modeling](#)
- [S-values](#)
- [Generating Tables](#)
- [Troubleshooting](#)
- [Consonance Functions for Linear Mixed-Effects Models](#)
- [Wish List](#)

Author(s)

Zad Rafi, Andrew D. Vigotsky

References

- Rafi, Z., and Greenland, S. (2020), "Semantic and Cognitive Tools to Aid Statistical Science: Replace Confidence and Significance by Compatibility and Surprise" *BMC Medical Research Methodology* <https://doi.org/10.1186/s12874-020-01105-9>
- Fraser DAS. The P-value function and statistical inference. *The American Statistician*. 2019;73(sup1):135-147. doi:10.1080/00031305.2018.1556735 <https://doi.org/10.1080/00031305.2018.1556735>
- Fraser DAS. P-Values: The Insight to Modern Statistical Inference. *Annual Review of Statistics and Its Application*. 2017;4(1):1-14. <https://doi.org/10.1146/annurev-statistics-060116-054139>
- Poole C. Beyond the confidence interval. *American Journal of Public Health*. 1987;77(2):195-199. doi:10.2105/AJPH.77.2.195 <https://doi.org/10.1002/jrsm.1410>
- Poole C. Confidence intervals exclude nothing. *American Journal of Public Health*. 1987;77(4):492-493. doi:10.2105/ajph.77.4.492 <https://doi.org/10.2105/ajph.77.4.492>
- Schweder T, Hjort NL. Confidence and Likelihood*. *Scandinavian Journal of Statistics*. 2002;29(2):309-332. doi:10.1111/1467-9469.00285 <https://doi.org/10.1111/1467-9469.00285>
- Schweder T, Hjort NL. Confidence, Likelihood, Probability: Statistical Inference with Confidence Distributions. Cambridge University Press; 2016. https://books.google.com/books/about/Confidence_Likelihood_Probability.html?id=t7KzCwAAQBAJ
- Singh K, Xie M, Strawderman WE. Confidence distribution (CD) – distribution estimator of a parameter. arXiv. August 2007. <http://arxiv.org/abs/0708.0976>
- Sullivan KM, Foster DA. Use of the confidence interval function. *Epidemiology*. 1990;1(1):39-42. doi:10.1097/00001648-199001000-00009 <https://doi.org/10.1097/00001648-199001000-00009>
- Whitehead J. The case for frequentism in clinical trials. *Statistics in Medicine*. 1993;12(15-16):1405-1413. doi:10.1002/sim.4780121506 <https://doi.org/10.1002/sim.4780121506>
- Xie M-g, Singh K. Confidence Distribution, the Frequentist Distribution Estimator of a Parameter: A Review. *International Statistical Review*. 2013;81(1):3-39. doi:10.1111/insr.12000 <https://doi.org/10.1111/insr.12000>
- Rothman KJ, Greenland S, Lash TL. Precision and statistics in epidemiologic studies. In: Rothman KJ, Greenland S, Lash TL, eds. *Modern Epidemiology*. 3rd ed. Lippincott Williams & Wilkins; 2008:148-167.
- Rücker G, Schwarzer G. Beyond the forest plot: The drapery plot. *Research Synthesis Methods*. April 2020. doi:10.1002/jrsm.1410 <https://doi.org/10.1002/jrsm.1410>
- Cox DR. Discussion. *International Statistical Review*. 2013;81(1):40-41. doi:10/gg9s2f <https://onlinelibrary.wiley.com/doi/abs/10.1111/insr.12007>

See Also

[curve_gen](#), [ggcurve](#), [curve_table](#)

curve_boot

Generate Consonance Functions via Bootstrapping

Description

Use the Bca bootstrap method and the t-bootstrap method from the `bcaboot` and `boot` packages to generate consonance distributions.

Usage

```
curve_boot(data = data, func = func, method = "bca", t0, tt, bb,
  replicates = 2000, steps = 1000, cores = getOption("mc.cores", 1L),
  table = TRUE)
```

Arguments

data	Dataset that is being used to create a consonance function.
func	Custom function that is used to create parameters of interest that will be bootstrapped.
method	The bootstrap method that will be used to generate the functions. Methods include "bca" which is the default, "bcapar", which is parametric bootstrapping using the bca method and "t", for the t-bootstrap/percentile method.
t0	Only used for the "bcapar" method. Observed estimate of theta, usually by maximum likelihood.
tt	Only used for the "bcapar" method. A vector of parametric bootstrap replications of theta of length B, usually large, say B = 2000
bb	Only used for the "bcapar" method. A B by p matrix of natural sufficient vectors, where p is the dimension of the exponential family.
replicates	Indicates how many bootstrap replicates are to be performed. The default is currently 20000 but more may be desirable, especially to make the functions more smooth.
steps	Indicates how many consonance intervals are to be calculated at various levels. For example, setting this to 100 will produce 100 consonance intervals from 0 to 100. Setting this to 10000 will produce more consonance levels. By default, it is set to 1000. Increasing the number substantially is not recommended as it will take longer to produce all the intervals and store them into a dataframe.
cores	Select the number of cores to use in order to compute the intervals The default is 1 core.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 7 items where the dataframe of standard values is in the first list and the table for it in the second if table = TRUE. The Bca intervals and table are found in the third and fourth list. The values for the density function are in the fifth object, while the Bca stats are in the sixth and seventh objects.

curve_compare

Compare Two Functions and Produces An AUC Score

Description

Compares the p-value/s-value, and likelihood functions and computes an AUC number.

Usage

```
curve_compare(data1, data2, type = "c", plot = TRUE, ...)
```

Arguments

data1	The first dataframe produced by one of the interval functions in which the intervals are stored.
data2	The second dataframe produced by one of the interval functions in which the intervals are stored.
type	Choose whether to plot a "consonance" function, a "surprisal" function or "likelihood". The default option is set to "c". The type must be set in quotes, for example <code>curve_compare(type = "s")</code> or <code>curve_compare(type = "c")</code> . Other options include "pd" for the consonance distribution function, and "cd" for the consonance density function, "l1" for relative likelihood, "l2" for log-likelihood, "l3" for likelihood and "d" for deviance function.
plot	by default it is set to TRUE and will use the <code>plot_compare()</code> function to plot the two functions.
...	Can be used to pass further arguments to <code>plot_compare()</code> .

Value

Computes an AUC score and returns a plot that graphs two functions.

See Also

[plot_compare\(\)](#)
[ggcurve\(\)](#)
[curve_table\(\)](#)

Examples

```
## Not run:
library(concurve)
GroupA <- rnorm(50)
GroupB <- rnorm(50)
RandomData <- data.frame(GroupA, GroupB)
intervalsdf <- curve_mean(GroupA, GroupB, data = RandomData)
GroupA2 <- rnorm(50)
GroupB2 <- rnorm(50)
RandomData2 <- data.frame(GroupA2, GroupB2)
model <- lm(GroupA2 ~ GroupB2, data = RandomData2)
randomframe <- curve_gen(model, "GroupB2")
curve_compare(intervalsdf[[1]], randomframe[[1]])

## End(Not run)
```

Description

Computes consonance intervals to produce P- and S-value functions for correlational analyses using the `cor.test` function in base R and places the interval limits for each interval level into a data frame along with the corresponding p-values and s-values.

Usage

```
curve_corr(x, y, alternative, method, steps = 10000,
           cores = getOption("mc.cores", 1L), table = TRUE)
```

Arguments

x	A vector that contains the data for one of the variables that will be analyzed for correlational analysis.
y	A vector that contains the data for one of the variables that will be analyzed for correlational analysis.
alternative	Indicates the alternative hypothesis and must be one of "two.sided", "greater" or "less". You can specify just the initial letter. "greater" corresponds to positive association, "less" to negative association.
method	A character string indicating which correlation coefficient is to be used for the test. One of "pearson", "kendall", or "spearman", can be abbreviated.
steps	Indicates how many consonance intervals are to be calculated at various levels. For example, setting this to 100 will produce 100 consonance intervals from 0 to 100. Setting this to 10000 will produce more consonance levels. By default, it is set to 1000. Increasing the number substantially is not recommended as it will take longer to produce all the intervals and store them into a dataframe.
cores	Select the number of cores to use in order to compute the intervals. The default is 1 core.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 3 items where the dataframe of values is in the first object, the values needed to calculate the density function in the second, and the table for the values in the third if `table = TRUE`.

Examples

```
## Not run:
GroupA <- rnorm(50)
GroupB <- rnorm(50)
joe <- curve_corr(x = GroupA, y = GroupB, alternative = "two.sided", method = "pearson")

## End(Not run)
```

curve_from_ratio	<i>Construct Consonance Function from Ratio Estimate</i>
------------------	--

Description

Convenience function to construct consonance functions from ratio measures (odds ratios, hazard ratios, risk ratios) given a point estimate and confidence interval bounds.

Usage

```
curve_from_ratio(ratio, lower, upper, conf.level = 0.95, steps = 1000,
  cores = getOption("mc.cores", 1L), table = TRUE)
```

Arguments

ratio	Point estimate of the ratio.
lower	Lower bound of the confidence interval.
upper	Upper bound of the confidence interval.
conf.level	Confidence level of the provided interval. Default is 0.95.
steps	Number of consonance levels to compute. Default is 1000.
cores	Number of cores for parallel computation.
table	Logical. If TRUE (default), includes a summary table.

Details

This is a convenience wrapper around [curve_rev](#) specifically for ratio measures. It automatically handles the log transformation and sets appropriate measure type.

Value

A list with class "concurve" containing the intervals dataframe, density dataframe, and optionally a summary table.

See Also

[curve_from_se\(\)](#) for constructing from standard error
[curve_rev\(\)](#) for the underlying function

Examples

```
## Not run:
# From a published odds ratio: OR = 1.5, 95% CI [1.1, 2.0]
result <- curve_from_ratio(ratio = 1.5, lower = 1.1, upper = 2.0)
ggcurve(result[[1]], type = "c", nullvalue = TRUE)

# Hazard ratio from survival analysis: HR = 0.75, 95% CI [0.60, 0.95]
result <- curve_from_ratio(ratio = 0.75, lower = 0.60, upper = 0.95)
ggcurve(result[[1]], type = "c", nullvalue = TRUE)

## End(Not run)
```

curve_from_se	<i>Construct Consonance Function from Standard Error</i>
---------------	--

Description

Convenience function to construct consonance functions given a point estimate and its standard error.

Usage

```
curve_from_se(estimate, se, measure = "mean", steps = 1000,
  cores = getOption("mc.cores", 1L), table = TRUE)
```

Arguments

estimate	Point estimate.
se	Standard error of the estimate.
measure	Type of measure: "mean" for differences (default), "ratio" for ratio measures (estimate should be on original scale).
steps	Number of consonance levels to compute. Default is 1000.
cores	Number of cores for parallel computation.
table	Logical. If TRUE (default), includes a summary table.

Details

This function constructs consonance intervals using the normal approximation: $\text{estimate} \pm z * \text{se}$ where z varies across confidence levels.

For ratio measures, the function handles the log transformation internally.

Value

A list with class "concurve" containing the intervals dataframe, density dataframe, and optionally a summary table.

See Also

[curve_from_ratio\(\)](#) for constructing from CI bounds

[curve_rev\(\)](#) for the underlying function

Examples

```
## Not run:
# Mean difference with SE
result <- curve_from_se(estimate = 2.5, se = 0.8, measure = "mean")
ggcurve(result[[1]], type = "c", nullvalue = TRUE)

# Odds ratio with SE (on log scale internally)
result <- curve_from_se(estimate = 1.5, se = 0.2, measure = "ratio")
ggcurve(result[[1]], type = "c", nullvalue = TRUE)

## End(Not run)
```

curve_lik	<i>Compute Profile Likelihood Functions</i>
-----------	---

Description

Compute Profile Likelihood Functions

Usage

```
curve_lik(likobject, data, table = TRUE)
```

Arguments

likobject	An object from the ProfileLikelihood package
data	The dataframe that was used to create the likelihood object in the ProfileLikelihood package.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 2 items where the dataframe of values is in the first object, and the table for the values in the second if table = TRUE.

Examples

```
library(ProfileLikelihood)
data(dataglm)
xx <- profilelike.glm(y ~ x1 + x2, dataglm, profile.theta = "group", binomial("logit"))
lik <- curve_lik(xx, dataglm)
```

curve_lmer	<i>Consonance Functions For Linear & Non-Linear Mixed-Effects Models.</i>
------------	---

Description

Computes thousands of consonance (confidence) intervals for the chosen parameter in the selected lme4 model and places the interval limits for each interval level into a data frame along with the corresponding p-values and s-values.. It is generally recommended to wrap this function using suppressMessages() due to the long list of profiling messages

Usage

```
curve_lmer(object, parm, method = "profile", zeta = NULL, nsim = NULL,
  FUN = NULL, boot.type = NULL, steps = 1000,
  cores = getOption("mc.cores", 1L), table = FALSE)
```

Arguments

object	The statistical model of interest from lme4 is to be indicated here.
parm	The variable of interest from the model (coefficients, intercept) for which the intervals are to be produced.
method	Chooses the method to be used to calculate the consonance intervals. There are currently four methods: "default", "wald", "lm", and "boot". The "default" method uses the profile likelihood method to compute intervals and can be used for models created by the 'lm' function. The "wald" method is typically what most people are familiar with when computing intervals based on the calculated standard error. The "lm" method allows this function to be used for specific scenarios like logistic regression and the 'glm' function. The "boot" method allows for bootstrapping at certain levels.
zeta	(for method = "profile" only:) likelihood cutoff (if not specified, as by default, computed from level).
nsim	number of simulations for parametric bootstrap intervals.
FUN	function; if NULL, an internal function that returns the fixed-effect parameters as well as the random-effect parameters on the standard deviation/correlation scale will be used.
boot.type	bootstrap confidence interval type, as described in boot.ci. Methods stud and bca are unavailable because they require additional components to be calculated.
steps	Indicates how many consonance intervals are to be calculated at various levels. For example, setting this to 100 will produce 100 consonance intervals from 0 to 100. Setting this to 10000 will produce more consonance levels. By default, it is set to 1000. Increasing the number substantially is not recommended as it will take longer to produce all the intervals and store them into a dataframe.
cores	Select the number of cores to use in order to compute the intervals. The default is 1 core.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 3 items where the dataframe of values is in the first object, the values needed to calculate the density function in the second, and the table for the values in the third if table = TRUE.

curve_mean

Consonance Functions For Mean Differences

Description

Computes thousands of consonance (confidence) intervals for the chosen parameter in a statistical test that compares means and places the interval limits for each interval level into a data frame along with the corresponding p-values and s-values.

Usage

```
curve_mean(x, y, data, paired = F, method = "default", replicates = 1000,
  steps = 10000, cores = getOption("mc.cores", 1L), table = TRUE)
```

Arguments

x	Variable that contains the data for the first group being compared.
y	Variable that contains the data for the second group being compared.
data	Data frame from which the variables are being extracted from.
paired	Indicates whether the statistical test is a paired difference test. By default, it is set to "F", which means the function will be an unpaired statistical test comparing two independent groups. Inserting "paired" will change the test to a paired difference test.
method	By default this is turned off (set to "default"), but allows for bootstrapping if "boot" is inserted into the function call.
replicates	Indicates how many bootstrap replicates are to be performed. The default is currently 20000 but more may be desirable, especially to make the functions more smooth.
steps	Indicates how many consonance intervals are to be calculated at various levels. For example, setting this to 100 will produce 100 consonance intervals from 0 to 100. Setting this to 10000 will produce more consonance levels. By default, it is set to 1000. Increasing the number substantially is not recommended as it will take longer to produce all the intervals and store them into a dataframe.
cores	Select the number of cores to use in order to compute the intervals. The default is 1 core.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 3 items where the dataframe of values is in the first object, the values needed to calculate the density function in the second, and the table for the values in the third if table = TRUE.

Examples

```
## Not run:
# Simulate random data
GroupA <- runif(100, min = 0, max = 100)
GroupB <- runif(100, min = 0, max = 100)
RandomData <- data.frame(GroupA, GroupB)
bob <- curve_mean(GroupA, GroupB, RandomData)

## End(Not run)
```

curve_meta

Consonance Functions For Meta-Analytic Data

Description

Computes thousands of consonance (confidence) intervals for the chosen parameter in the meta-analysis done by the metafor package and places the interval limits for each interval level into a data frame along with the corresponding p-values and s-values.

Usage

```
curve_meta(x, measure = "default", method = "uni", parm = NULL,
  robust = FALSE, cluster = NULL, adjust = FALSE, steps = 1000,
  cores = getOption("mc.cores", 1L), table = TRUE)
```

Arguments

x	Object where the meta-analysis parameters are stored, typically a list produced by 'metafor'
measure	Indicates whether the object has a log transformation or is normal/default. The default setting is "default. If the measure is set to "ratio", it will take logarithmically transformed values and convert them back to normal values in the dataframe. This is typically a setting used for binary outcomes such as risk ratios, hazard ratios, and odds ratios.
method	Indicates which meta-analysis metafor function is being used. Currently supports rma.uni ("uni"), which is the default, rma.mh ("mh"), and rma.peto ("peto")
parm	Typically ignored, but needed sometimes in order to specify which variable to produce function for.
robust	a logical indicating whether to produce cluster robust interval estimates Default is FALSE.
cluster	a vector specifying a clustering variable to use for constructing the sandwich estimator of the variance-covariance matrix. Default setting is NULL.
adjust	logical indicating whether a small-sample correction should be applied to the variance-covariance matrix. Default is FALSE.
steps	Indicates how many consonance intervals are to be calculated at various levels. For example, setting this to 100 will produce 100 consonance intervals from 0 to 100. Setting this to 10000 will produce more consonance levels. By default, it is set to 1000. Increasing the number substantially is not recommended as it will take longer to produce all the intervals and store them into a dataframe
cores	Select the number of cores to use in order to compute the intervals The default is 1 core.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 3 items where the dataframe of values is in the first object, the values needed to calculate the density function in the second, and the table for the values in the third if table = TRUE.

Examples

```
## Not run:
# Simulate random data for two groups in two studies
GroupAData <- runif(20, min = 0, max = 100)
GroupAMean <- round(mean(GroupAData), digits = 2)
GroupASD <- round(sd(GroupAData), digits = 2)

GroupBData <- runif(20, min = 0, max = 100)
GroupBMean <- round(mean(GroupBData), digits = 2)
GroupBSD <- round(sd(GroupBData), digits = 2)
```

```

GroupCData <- runif(20, min = 0, max = 100)
GroupCMean <- round(mean(GroupCData), digits = 2)
GroupCSD <- round(sd(GroupCData), digits = 2)

GroupDData <- runif(20, min = 0, max = 100)
GroupDMean <- round(mean(GroupDData), digits = 2)
GroupDSD <- round(sd(GroupDData), digits = 2)

# Combine the data

StudyName <- c("Study1", "Study2")
MeanTreatment <- c(GroupAMean, GroupCMean)
MeanControl <- c(GroupBMean, GroupDMean)
SDTreatment <- c(GroupASD, GroupCSD)
SDControl <- c(GroupBSD, GroupDSD)
NTreatment <- c(20, 20)
NControl <- c(20, 20)

metadf <- data.frame(
  StudyName, MeanTreatment, MeanControl,
  SDTreatment, SDControl, NTreatment, NControl
)

# Use metafor to calculate the standardized mean difference

library(metafor)

dat <- escalc(
  measure = "SMD", m1i = MeanTreatment, sd1i = SDTreatment,
  n1i = NTreatment, m2i = MeanControl, sd2i = SDControl,
  n2i = NControl, data = metadf
)

# Pool the data using a particular method. Here "FE" is the fixed-effects model

res <- rma(yi, vi,
  data = dat, slab = paste(StudyName, sep = ", "),
  method = "FE", digits = 2
)

# Calculate the intervals using the metainterval function

metaf <- curve_meta(res)

## End(Not run)

```

Description

A convenience wrapper around [curve_wrap](#) for common model objects. Automatically selects the appropriate confidence interval method based on the model class or user specification.

Usage

```
curve_model(model, param, method = "default", steps = 1000,
            cores = getOption("mc.cores", 1L), table = TRUE)
```

Arguments

model	A fitted model object (lm, glm, nls, lme, etc.)
param	Character string specifying which parameter to extract.
method	Method for computing confidence intervals: "default" Uses <code>confint.default()</code> - Wald/normal approximation (fast) "profile" Uses <code>confint()</code> - profile likelihood (slower, more accurate for GLMs)
steps	Number of consonance levels to compute. Default is 1000.
cores	Number of cores for parallel computation.
table	Logical. If TRUE (default), includes a summary table.

Details

This is a convenience function that constructs the appropriate `ci_func` for [curve_wrap](#) based on the specified method. For maximum flexibility, use `curve_wrap` directly.

Method selection guidelines:

- **Linear models (lm):** "default" is appropriate and fast
- **GLMs:** "profile" is more accurate, especially for small samples
- **Mixed models:** "profile" recommended but can be slow
- **Robust models (rlm):** "default" typically used

Value

A list with class "concurve" (see [curve_wrap](#) for details).

See Also

[curve_wrap\(\)](#) for the generic wrapper
[curve_gen\(\)](#) for the original model-based function
[ggcurve\(\)](#) for plotting

Examples

```
## Not run:
# Linear model - Wald intervals (fast)
model <- lm(mpg ~ wt + hp, data = mtcars)
result <- curve_model(model, param = "wt", method = "default")
ggcurve(result[[1]], type = "c")

# GLM - Profile likelihood intervals (more accurate)
model <- glm(am ~ wt, data = mtcars, family = binomial)
result <- curve_model(model, param = "wt", method = "profile")

# Compare methods
wald <- curve_model(model, param = "wt", method = "default")
profile <- curve_model(model, param = "wt", method = "profile")
```

```
# Plot both to see the difference

## End(Not run)
```

curve_overlap

Calculate Overlap Between Consonance Functions

Description

Quantifies the area of overlap between two consonance functions, providing a measure of compatibility between two estimates.

Usage

```
curve_overlap(data1, data2, type = "c", plot = TRUE,
              title = "Consonance Function Overlap")
```

Arguments

data1	First concurve object or intervals data frame.
data2	Second concurve object or intervals data frame.
type	Function type for comparison: "c" for consonance (default), "s" for surprisal.
plot	Logical. If TRUE (default), displays overlap visualization.
title	Plot title.

Details

This function computes the overlap between two consonance functions, which provides a measure of how compatible two estimates are with each other. Higher overlap indicates greater compatibility.

The overlap is calculated by finding the intersection of intervals at each confidence level and integrating over the shared region.

Value

A list containing:

overlap_area Estimated area of overlap

total_area Total combined area

overlap_ratio Ratio of overlap to total area

max_shared_level Maximum confidence level where intervals overlap

See Also

[curve_compare\(\)](#) for graphical comparison

[plot_compare\(\)](#) for visualization

Examples

```
## Not run:
# Compare two study results
study1 <- curve_from_ratio(1.5, 1.1, 2.0)
study2 <- curve_from_ratio(1.3, 0.9, 1.8)

overlap <- curve_overlap(study1[[1]], study2[[1]])
print(overlap)

## End(Not run)
```

curve_rev

Reverse Engineer Consonance / Likelihood Functions Using the Point Estimate and Confidence Limits

Description

Using the confidence limits and point estimates from a dataset, one can use these estimates to compute thousands of consonance intervals and graph the intervals to form a consonance, surprisal, and likelihood functions. The intervals are calculated from the approximated normal distribution, however, users should be cautious as this function is currently designed for similar situations (involving ratios and normal approximations), nevertheless the function also works for means but should be used skeptically, as it can break down in many situations and give implausible numbers. Computations of likelihood functions for means is currently not supported.

Usage

```
curve_rev(point, LL = NULL, UL = NULL, se = NULL, conf.level = 0.95,
  type = "c", measure = "ratio", steps = 10000,
  cores = getOption("mc.cores", 1L), table = TRUE)
```

Arguments

point	The point estimate from an analysis. Ex: 1.20
LL	The lower confidence limit from an analysis Ex: 1.0
UL	The upper confidence limit from an analysis Ex: 1.4
se	The standard error of the point estimate. Ex: 0.05
conf.level	Confidence level of the interval estimate.
type	Indicates whether the produced result should be a consonance function or a likelihood function. The default is "c" for consonance and likelihood can be set via "l".
measure	The type of data being used. If they involve mean differences, then the "mean" option should be used. If the data are ratios, then the "ratio" option should be used. "ratio" is currently the default option. Currently, this function is designed to be used with ratios and normal approximations rather than means.
steps	Indicates how many consonance intervals are to be calculated at various levels. For example, setting this to 100 will produce 100 consonance intervals from 0 to 100. Setting this to 10000 will produce more consonance levels. By default, it is set to 1000. Increasing the number substantially is not recommended as it will take longer to produce all the intervals and store them into a dataframe.

cores	Select the number of cores to use in order to compute the intervals The default is 1 core.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 3 items where the dataframe of values is in the first object, the values needed to calculate the density function in the second, and the table for the values in the third if table = TRUE.

See Also

ggcurve()
curve_compare()
plot_compare()

Examples

```
## Not run:
# From a real published study. Point estimate of the result was hazard ratio of 1.61 and
# lower bound of the interval is 0.997 while upper bound of the interval is 2.59.
#
df <- curve_rev(point = 1.61, LL = 0.997, UL = 2.59, measure = "ratio")

## End(Not run)
```

curve_snowflake

Construct Consonance Functions from Snowflake Query Results

Description

Connects to a Snowflake database and constructs consonance distributions from query results containing point estimates and confidence intervals.

Usage

```
curve_snowflake(conn, query, estimate_col = "estimate",
  lower_col = "lower", upper_col = "upper", conf.level = 0.95,
  steps = 1000, cores = getOption("mc.cores", 1L), table = TRUE)
```

Arguments

conn	A DBI connection object to Snowflake database.
query	SQL query string that returns columns for estimate, lower, and upper bounds.
estimate_col	Name of the column containing point estimates. Default is "estimate".
lower_col	Name of the column containing lower bounds. Default is "lower".
upper_col	Name of the column containing upper bounds. Default is "upper".
conf.level	Confidence level of the input intervals. Default is 0.95.
steps	Number of consonance levels to compute. Default is 1000.
cores	Number of cores for parallel computation.
table	Logical. If TRUE (default), includes a summary table.

Details

This function requires the DBI and odbc packages to be installed. It connects to Snowflake, executes the provided query, and constructs consonance functions from the results.

Value

A list with class "concurve" containing the intervals dataframe, density dataframe, and optionally a summary table.

See Also

[curve_snowflake_batch\(\)](#) for batch processing
[curve_rev\(\)](#) for constructing curves from published intervals
[export_for_powerbi\(\)](#) for exporting results

Examples

```
## Not run:
library(DBI)
library(odbc)

# Connect to Snowflake
conn <- dbConnect(odbc::odbc(),
  Driver = "Snowflake",
  Server = "your_account.snowflakecomputing.com",
  Database = "your_database",
  Schema = "your_schema",
  UID = "your_username",
  PWD = "your_password"
)

# Query with estimate and CI columns
query <- "SELECT estimate, lower_ci, upper_ci FROM results WHERE analysis_id = 1"
result <- curve_snowflake(conn, query,
  estimate_col = "estimate",
  lower_col = "lower_ci",
  upper_col = "upper_ci"
)

ggcurve(result[[1]], type = "c")

dbDisconnect(conn)

## End(Not run)
```

curve_snowflake_batch *Batch Process Multiple Snowflake Queries for Consonance Functions*

Description

Executes multiple queries against a Snowflake database and constructs consonance distributions for each result set.

Usage

```
curve_snowflake_batch(conn, queries, estimate_col = "estimate",
  lower_col = "lower", upper_col = "upper", conf.level = 0.95,
  steps = 1000, cores = getOption("mc.cores", 1L), table = TRUE)
```

Arguments

conn	A DBI connection object to Snowflake database.
queries	A named list of SQL query strings.
estimate_col	Name of the column containing point estimates.
lower_col	Name of the column containing lower bounds.
upper_col	Name of the column containing upper bounds.
conf.level	Confidence level of the input intervals. Default is 0.95.
steps	Number of consonance levels to compute. Default is 1000.
cores	Number of cores for parallel computation.
table	Logical. If TRUE (default), includes summary tables.

Value

A named list of concurve objects, one for each query.

See Also

[curve_snowflake\(\)](#) for single query processing

[plot_compare\(\)](#) for comparing results

Examples

```
## Not run:
queries <- list(
  "Model A" = "SELECT estimate, lower_ci, upper_ci FROM results WHERE model = 'A'",
  "Model B" = "SELECT estimate, lower_ci, upper_ci FROM results WHERE model = 'B'"
)

results <- curve_snowflake_batch(conn, queries,
  estimate_col = "estimate",
  lower_col = "lower_ci",
  upper_col = "upper_ci"
)

# Plot comparison
plot_compare(results[["Model A"]][[1]], results[["Model B"]][[1]])

## End(Not run)
```

curve_summary

*Generate Summary Statistics for Consonance Objects***Description**

Produces a summary of key statistics from a consonance function.

Usage

```
curve_summary(data, levels = c(0.5, 0.9, 0.95, 0.99), null_value = NULL,
  digits = 4)
```

Arguments

data	A concurve object or intervals data frame.
levels	Confidence levels to include in summary. Default is c(0.50, 0.90, 0.95, 0.99).
null_value	Reference value for compatibility assessment. Default is 0 for differences, 1 for ratios.
digits	Number of decimal places in output. Default is 4.

Details

This function provides a summary of a consonance function including:

- Point estimate (value at maximum consonance)
- Confidence intervals at specified levels
- P-value and S-value at the null hypothesis
- Interval width as a measure of precision

Value

A data frame with summary statistics.

See Also

[curve_table\(\)](#) for formatted interval tables

Examples

```
## Not run:
model <- lm(mpg ~ wt, data = mtcars)
result <- curve_gen(model, "wt")

# Get summary
curve_summary(result[[1]])

# Custom levels
curve_summary(result[[1]], levels = c(0.80, 0.95))

## End(Not run)
```

curve_surv

*Consonance Functions For Survival Data***Description**

Computes thousands of consonance (confidence) intervals for the chosen parameter in the Cox model computed by the 'survival' package and places the interval limits for each interval level into a data frame along with the corresponding p-value and s-value.

Usage

```
curve_surv(data, x, steps = 10000, cores = getOption("mc.cores", 1L),
  table = TRUE)
```

Arguments

data	Object where the Cox model is stored, typically a list produced by the 'survival' package.
x	Predictor of interest within the survival model for which the consonance intervals should be computed.
steps	Indicates how many consonance intervals are to be calculated at various levels. For example, setting this to 100 will produce 100 consonance intervals from 0 to 100. Setting this to 10000 will produce more consonance levels. By default, it is set to 1000. Increasing the number substantially is not recommended as it will take longer to produce all the intervals and store them into a dataframe.
cores	Select the number of cores to use in order to compute the intervals The default is 1 core.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 3 items where the dataframe of values is in the first object, the values needed to calculate the density function in the second, and the table for the values in the third if table = TRUE.

Examples

```
## Not run:
library(carData)
Rossi[1:5, 1:10]
library(survival)

mod.allison <- coxph(Surv(week, arrest) ~ fin + age + race + wexp + mar + paro + prio,
  data = Rossi
)
mod.allison

z <- curve_surv(mod.allison, "prio")

## End(Not run)
```

curve_table

*Produce Tables For concurve Functions***Description**

Produces publication-ready tables with relevant statistics of interest for functions produced from the concurve package.

Usage

```
curve_table(data, levels, type = "c", format = "data.frame")
```

Arguments

data	Dataframe from a concurve function to produce a table for
levels	Levels of the consonance intervals or likelihood intervals that should be included in the table.
type	Indicates whether the table is for a consonance function or likelihood function. The default is set to "c" for consonance and can be switched to "l" for likelihood.
format	The format of the tables. The options include "data.frame" which is the default, "docx" (which creates a table for a word document), "pptx" (which creates a table for powerpoint), "latex", (which creates a table for a TeX document), and "image", which produces an image of the table.

See Also

[ggcurve\(\)](#)
[curve_compare\(\)](#)
[plot_compare\(\)](#)

Examples

```
## Not run:
library(concurve)

GroupA <- rnorm(500)
GroupB <- rnorm(500)

RandomData <- data.frame(GroupA, GroupB)

intervalsdf <- curve_mean(GroupA, GroupB, data = RandomData, method = "default")

(z <- curve_table(intervalsdf[[1]], format = "data.frame"))
(z <- curve_table(intervalsdf[[1]], format = "latex"))
(z <- curve_table(intervalsdf[[1]], format = "image"))

## End(Not run)
```

curve_wrap

*Construct Consonance Functions from Any CI-Producing Function***Description**

A generic wrapper that constructs consonance, surprisal, and likelihood functions from any function that produces confidence intervals. This is the most flexible approach for generating consonance curves from arbitrary statistical procedures.

Usage

```
curve_wrap(ci_func, steps = 1000, cores = getOption("mc.cores", 1L),
  table = TRUE)
```

Arguments

ci_func	A function that takes a confidence level (0-1) as its first argument and returns a numeric vector of length 2 (lower, upper bounds). See examples for common patterns.
steps	Number of consonance levels to compute. Default is 1000.
cores	Number of cores for parallel computation. Default uses <code>getOption("mc.cores", 1L)</code> .
table	Logical. If TRUE (default), includes a summary table of key intervals in the output.

Details

This function repeatedly calls `ci_func` at different confidence levels to construct the full consonance function. The `ci_func` argument must be a function that:

1. Takes a confidence level (numeric between 0 and 1) as its first argument
2. Returns a numeric vector of exactly length 2: `c(lower_bound, upper_bound)`

Common patterns for `ci_func`:

- Linear models: `function(level) confint.default(model, parm = "x", level = level)`
- GLMs (profile): `function(level) confint(model, parm = "x", level = level)`
- t-tests: `function(level) t.test(x, conf.level = level)$conf.int`
- Correlations: `function(level) cor.test(x, y, conf.level = level)$conf.int`
- Proportions: `function(level) prop.test(x, n, conf.level = level)$conf.int`
- Survival: `function(level) exp(confint(coxph_model, level = level)["var",])`

Value

A list with class "concurve" containing:

Intervals Dataframe Data frame with columns: `lower.limit`, `upper.limit`, `intrvl.width`, `intrvl.level`, `cdf`, `pvalue`, `svalue`

Intervals Density Data frame for plotting density functions

Intervals Table Summary table of key intervals (if `table = TRUE`)

See Also

[ggcurve\(\)](#) for plotting
[curve_gen\(\)](#) for model-specific consonance functions
[curve_rev\(\)](#) for constructing curves from published intervals
[curve_table\(\)](#) for interval summaries

Examples

```
## Not run:
# Example 1: Linear model
data(mtcars)
model <- lm(mpg ~ wt + hp, data = mtcars)
ci_func <- function(level) confint.default(model, parm = "wt", level = level)
result <- curve_wrap(ci_func)
ggcurve(result[[1]], type = "c")

# Example 2: t-test
x <- rnorm(30, mean = 5)
ci_func <- function(level) t.test(x, conf.level = level)$conf.int
result <- curve_wrap(ci_func)
ggcurve(result[[1]], type = "c", nullvalue = 0)

# Example 3: Correlation
x <- rnorm(50)
y <- x + rnorm(50)
ci_func <- function(level) cor.test(x, y, conf.level = level)$conf.int
result <- curve_wrap(ci_func)

# Example 4: GLM with profile likelihood CIs
model <- glm(am ~ wt, data = mtcars, family = binomial)
ci_func <- function(level) confint(model, parm = "wt", level = level)
result <- curve_wrap(ci_func)

# Example 5: Proportion test
ci_func <- function(level) prop.test(45, 100, conf.level = level)$conf.int
result <- curve_wrap(ci_func)

# Example 6: Custom function with additional arguments
my_ci <- function(level, data, method) {
  cor.test(data$x, data$y, method = method, conf.level = level)$conf.int
}
df <- data.frame(x = rnorm(50), y = rnorm(50))
ci_func <- function(level) my_ci(level, data = df, method = "spearman")
result <- curve_wrap(ci_func)

## End(Not run)
```

Description

Exports consonance function data in formats optimized for Power BI visualization and analysis.

Usage

```
export_for_powerbi(data, file, format = NULL, include_metadata = TRUE,
  pivot = FALSE)
```

Arguments

<code>data</code>	A concurve object or intervals data frame.
<code>file</code>	Output file path. Supports .csv, .xlsx, or .json extensions.
<code>format</code>	Output format: "csv" (default), "excel", or "json". If NULL, inferred from file extension.
<code>include_metadata</code>	Logical. If TRUE, includes additional metadata columns useful for Power BI DAX measures. Default is TRUE.
<code>pivot</code>	Logical. If TRUE, creates a pivoted format with separate columns for lower and upper bounds at each confidence level. Default is FALSE.

Details

This function prepares consonance function data for import into Power BI. The exported data includes:

- Confidence levels and corresponding intervals
- P-values and S-values
- Interval widths for precision analysis

When `include_metadata = TRUE`, adds columns for:

- `ci_label`: Human-readable confidence level (e.g., "95\
- `significance`: Whether interval excludes null value

Value

Invisibly returns the exported data frame.

See Also

[curve_table\(\)](#) for formatted interval tables

[curve_snowflake\(\)](#) for Snowflake database integration

Examples

```
## Not run:
# Generate consonance data
model <- lm(mpg ~ wt, data = mtcars)
result <- curve_gen(model, "wt")

# Export to CSV for Power BI
export_for_powerbi(result[[1]], "consonance_data.csv")

# Export with metadata
export_for_powerbi(result[[1]], "consonance_data.csv", include_metadata = TRUE)

# Export to Excel
```

```
export_for_powerbi(result[[1]], "consonance_data.xlsx", format = "excel")

## End(Not run)
```

ggcurve

Plots Consonance, Surprisal, and Likelihood Functions

Description

Takes the dataframe produced by the interval functions and plots the p-values/s-values, consonance (confidence) levels, and the interval estimates to produce a p-value/s-value function using ggplot2 graphics.

Usage

```
ggcurve(data, type = "c", measure = "default", levels = 0.95,
  nullvalue = NULL, position = "pyramid", title = "Consonance Function",
  subtitle = "The function displays intervals at every level.",
  xaxis = expression(theta == ~"Range of Values"),
  yaxis1 = expression(paste(italic(p), "-value")),
  yaxis2 = "Levels for CI (%)", color = darken("#009E73", 0.5),
  fill = "#239a98")
```

Arguments

data	The dataframe produced by one of the interval functions in which the intervals are stored.
type	Choose whether to plot a "consonance" function, a "surprisal" function or "likelihood". The default option is set to "c". The type must be set in quotes, for example ggcurve (type = "s") or ggcurve(type = "c"). Other options include "pd" for the consonance distribution function, and "cd" for the consonance density function, "l1" for relative likelihood, "l2" for log-likelihood, "l3" for likelihood and "d" for deviance function.
measure	Indicates whether the object has a log transformation or is normal/default. The default setting is "default". If the measure is set to "ratio", it will take logarithmically transformed values and convert them back to normal values in the dataframe. This is typically a setting used for binary outcomes and their measures such as risk ratios, hazard ratios, and odds ratios.
levels	Indicates which interval levels should be plotted on the function. By default it is set to 0.95 to plot the 95% interval on the consonance function, but more levels can be plotted by using the c() function for example, levels = c(0.50, 0.75, 0.95).
nullvalue	Indicates whether the null value for the measure should be plotted. By default, it is set to NULL, meaning it will not be plotted as a vertical line. Changing this to a numerical vector will specify the region where a line should be plotted or an area that should be shaded. The input must be a numerical vector, for example c(-0.5, 0.5) or a single numerical vector such as 0 or 1.

position	Determines the orientation of the P-value (consonance) function. By default, it is set to "pyramid", meaning the p-value function will stand right side up, like a pyramid. However, it can also be inverted via the option "inverted". This will also change the sequence of the y-axes to match the orientation. This can be set as such, <code>ggcurve(type = "c", data = df, position = "inverted")</code> .
title	A custom title for the graph. By default, it is set to "Consonance Function". In order to set a title, it must be in quotes. For example, <code>ggcurve(type = "c", data = x, title = "Custom Title")</code> .
subtitle	A custom subtitle for the graph. By default, it is set to "The function contains consonance/confidence intervals at every level and the P-values." In order to set a subtitle, it must be in quotes. For example, <code>ggcurve(type = "c", data = x, subtitle = "Custom Subtitle")</code> .
xaxis	A custom x-axis title for the graph. By default, it is set to "Range of Values. In order to set a x-axis title, it must be in quotes. For example, <code>ggcurve(type = "c", data = x, xaxis = "Hazard Ratio")</code> .
yaxis1	A custom y-axis title for the graph. By default, it is set to "Consonance Level". In order to set a y-axis title, it must be in quotes. For example, <code>ggcurve(type = "c", data = x, yaxis1 = "Confidence Level")</code> .
yaxis2	A custom y-axis title for the graph. By default, it is set to "Levels for CI". In order to set a y-axis title, it must be in quotes. For example, <code>ggcurve(type = "c", data = x, yaxis2 = "Confidence Level")</code> .
color	Item that allows the user to choose the color of the points and the ribbons in the graph. By default, it is set to <code>color = "#555555"</code> . The inputs must be in quotes. For example, <code>ggcurve(type = "c", data = x, color = "#333333")</code> .
fill	Item that allows the user to choose the color of the ribbons in the graph. By default, it is set to <code>fill = "#239a98"</code> . The inputs must be in quotes. For example, <code>ggcurve(type = "c", data = x, fill = "#333333")</code> .

Value

A plot with intervals at every consonance level graphed with their corresponding p-values and compatibility levels.

See Also

[plot_compare\(\)](#)

Examples

```
## Not run:
# Simulate random data

library(concurve)

GroupA <- rnorm(500)
GroupB <- rnorm(500)

RandomData <- data.frame(GroupA, GroupB)

intervalsdf <- suppressMessages(curve_mean(GroupA, GroupB, data = RandomData, method = "default"))
ggcurve(type = "c", intervalsdf[[1]], nullvalue = c(0))

## End(Not run)
```

plot_compare	<i>Graph and Compare Consonance, Surprisal, and Likelihood Functions</i>
--------------	--

Description

Compares the p-value/s-value, and likelihood functions using ggplot2 graphics.

Usage

```
plot_compare(data1, data2, type = "c", measure = "default",
  nullvalue = FALSE, position = "pyramid", title = "Interval Functions",
  subtitle = "The function displays intervals at every level.",
  xaxis = expression(theta == ~"Range of Values"),
  yaxis1 = expression(paste(italic(p), "-value")),
  yaxis2 = "Levels for CI (%)", color1 = darken("#D55E00", 0.2),
  color2 = darken("#009E73", 0.2), fill1 = "#99c7c7", fill2 = "#d46c5b")
```

Arguments

data1	The first dataframe produced by one of the interval functions in which the intervals are stored.
data2	The second dataframe produced by one of the interval functions in which the intervals are stored.
type	Choose whether to plot a "consonance" function, a "surprisal" function or "likelihood". The default option is set to "c". The type must be set in quotes, for example plot_compare(type = "s") or plot_compare(type = "c"). Other options include "pd" for the consonance distribution function, and "cd" for the consonance density function, "l1" for relative likelihood, "l2" for log-likelihood, "l3" for likelihood and "d" for deviance function.
measure	Indicates whether the object has a log transformation or is normal/default. The default setting is "default". If the measure is set to "ratio", it will take logarithmically transformed values and convert them back to normal values in the dataframe. This is typically a setting used for binary outcomes and their measures such as risk ratios, hazard ratios, and odds ratios.
nullvalue	Indicates whether the null value for the measure should be plotted. By default, it is set to FALSE, meaning it will not be plotted as a vertical line. Changing this to TRUE, will plot a vertical line at 0 when the measure is set to "default" and a vertical line at 1 when the measure is set to "ratio". For example, plot_compare(type = "c", data = df, measure = "ratio", nullvalue = "present"). This feature is not yet available for surprisal functions.
position	Determines the orientation of the P-value (consonance) function. By default, it is set to "pyramid", meaning the p-value function will stand right side up, like a pyramid. However, it can also be inverted via the option "inverted". This will also change the sequence of the y-axes to match the orientation. This can be set as such, plot_compare(type = "c", data = df, position = "inverted").
title	A custom title for the graph. By default, it is set to "Consonance Function". In order to set a title, it must be in quotes. For example, plot_compare(type = "c", data = x, title = "Custom Title").

subtitle	A custom subtitle for the graph. By default, it is set to "The function contains consonance/confidence intervals at every level and the P-values." In order to set a subtitle, it must be in quotes. For example, <code>plot_compare(type = "c", data = x, subtitle = "Custom Subtitle")</code> .
xaxis	A custom x-axis title for the graph. By default, it is set to "Range of Values. In order to set a x-axis title, it must be in quotes. For example, <code>plot_compare(type = "c", data = x, xaxis = "Hazard Ratio")</code> .
yaxis1	A custom y-axis title for the graph. By default, it is set to "Consonance Level". In order to set a y-axis title, it must be in quotes. For example, <code>ggcurve(type = "c", data = x, yaxis1 = "Confidence Level")</code> .
yaxis2	A custom y-axis title for the graph. By default, it is set to "Levels for CI". In order to set a y-axis title, it must be in quotes. For example, <code>ggcurve(type = "c", data = x, yaxis2 = "Confidence Level")</code> .
color1	Item that allows the user to choose the color of the points and the ribbons in the graph. By default, it is set to <code>darken("#D55E00", 0.4)</code> . The inputs must be in quotes.
color2	Item that allows the user to choose the color of the points and the ribbons in the graph. By default, it is set to <code>darken("#009E73", 0.4)</code> . The inputs must be in quotes. For example, <code>plot_compare(type = "c", data = x, color = "#333333")</code> .
fill1	Item that allows the user to choose the color of the ribbons in the graph for data1. By default, it is set to <code>fill1 = "#239a98"</code> . The inputs must be in quotes. For example, <code>plot_compare(type = "c", data = x, fill1 = "#333333")</code> .
fill2	Item that allows the user to choose the color of the ribbons in the graph for data1. By default, it is set to <code>fill2 = "#d46c5b"</code> . The inputs must be in quotes. For example, <code>plot_compare(type = "c", data = x, fill2 = "#333333")</code> .

Value

A plot that compares two functions.

See Also

[ggcurve\(\)](#)
[curve_compare\(\)](#)

Examples

```
## Not run:
library(concurve)

GroupA <- rnorm(50)
GroupB <- rnorm(50)
RandomData <- data.frame(GroupA, GroupB)
intervalsdf <- curve_mean(GroupA, GroupB, data = RandomData)
GroupA2 <- rnorm(50)
GroupB2 <- rnorm(50)
RandomData2 <- data.frame(GroupA2, GroupB2)
model <- lm(GroupA2 ~ GroupB2, data = RandomData2)

randomframe <- curve_gen(model, "GroupB2")

plot_compare(intervalsdf[[1]], randomframe[[1]], type = "c")
```

```
## End(Not run)
```

plot_multi

Plot Multiple Consonance Functions

Description

Creates a combined plot showing multiple consonance functions for comparison.

Overlays multiple consonance or surprisal functions on a single plot for direct visual comparison of effect estimates across groups, time periods, or studies.

Usage

```
plot_multi(..., type = "c", measure = "default", nullvalue = NULL,
  position = "pyramid", title = "Comparison of Consonance Functions",
  subtitle = "Functions display intervals at every level.",
  xaxis = expression(theta == ~"Effect Size"),
  yaxis1 = expression(paste(italic(p), "-value")),
  yaxis2 = "Confidence Level (%)", colors = NULL, alpha = 0.15,
  legend.position = "bottom")
```

```
plot_multi(..., type = "c", measure = "default", nullvalue = NULL,
  position = "pyramid", title = "Comparison of Consonance Functions",
  subtitle = "Functions display intervals at every level.",
  xaxis = expression(theta == ~"Effect Size"),
  yaxis1 = expression(paste(italic(p), "-value")),
  yaxis2 = "Confidence Level (%)", colors = NULL, alpha = 0.15,
  legend.position = "bottom")
```

Arguments

...	Named concurve dataframes to compare. Names become legend labels. Alternatively, pass a single named list of dataframes.
type	Character. Type of function to plot: "c" for consonance (default), "s" for surprisal.
measure	Character. Scale type: "default" for linear, "ratio" for log scale (odds ratios, hazard ratios, etc.).
nullvalue	Numeric. Value(s) to mark with vertical reference line(s). Use single value (e.g., 0) or vector for range (e.g., c(-0.5, 0.5)).
position	Character. Orientation of consonance function: "pyramid" (default) or "inverted".
title	Character. Plot title.
subtitle	Character. Plot subtitle.
xaxis	Character or expression. X-axis label.
yaxis1	Character or expression. Primary y-axis label.
yaxis2	Character. Secondary y-axis label (for CI levels).

colors	Character vector. Colors for each curve. If NULL, uses colorblind-friendly palette.
alpha	Numeric. Transparency for ribbon fill (0-1). Default is 0.15.
legend.position	Character or numeric vector. Legend position. Default is "bottom".
labels	Character vector of labels for each curve. If NULL, uses names from input or generates sequential labels.

Details

This function overlays multiple consonance or surprisal functions on a single plot for easy comparison. Each curve is distinguished by color.

Value

A ggplot object.
A ggplot2 object.

See Also

[ggcurve\(\)](#) for single curve plotting
[curve_overlap\(\)](#) for quantifying overlap
[ggcurve\(\)](#), [plot_compare\(\)](#), [curve_compare\(\)](#)

Examples

```
## Not run:
# Compare three analyses
result1 <- curve_from_ratio(1.5, 1.1, 2.0)
result2 <- curve_from_ratio(1.3, 0.9, 1.8)
result3 <- curve_from_ratio(1.8, 1.2, 2.5)

plot_multi(result1[[1]], result2[[1]], result3[[1]],
  labels = c("Study A", "Study B", "Study C"),
  nullvalue = 1,
  title = "Comparison of Odds Ratios"
)

## End(Not run)

## Not run:
# Compare two studies
study1 <- curve_from_se(point = 0.5, se = 0.2, df = 50)
study2 <- curve_from_se(point = 0.8, se = 0.25, df = 80)

plot_multi(
  "Study A" = study1[[1]],
  "Study B" = study2[[1]],
  nullvalue = 0,
  title = "Comparison of Treatment Effects"
)

# Using a list
curves_list <- list("Group 1" = study1[[1]], "Group 2" = study2[[1]])
```



```
plot_multi( curves_list, type = "s", nullvalue = 0 )  
  
## End(Not run)
```

RobustMax	<i>Robust Max, an alternative to max() that doesn't throw a warning</i>
-----------	---

Description

Robust Max, an alternative to max() that doesn't throw a warning

Usage

```
RobustMax(x)
```

Arguments

x	A vector to find the maximum value of
---	---------------------------------------

Value

The max value from a vector

RobustMin	<i>Robust Min, an alternative to max() that doesn't throw a warning</i>
-----------	---

Description

Robust Min, an alternative to max() that doesn't throw a warning

Usage

```
RobustMin(x)
```

Arguments

x	A vector find the minimum value of
---	------------------------------------

Value

The minimum value from the vector

Index

concurve (concurve-package), 3
concurve-package, 3
curve_boot, 4
curve_compare, 5
curve_compare(), 16, 18, 23, 30, 32
curve_corr, 7
curve_from_ratio, 8
curve_from_ratio(), 9
curve_from_se, 9
curve_from_se(), 8
curve_gen, 4
curve_gen(), 15, 25
curve_lik, 10
curve_lmer, 10
curve_mean, 11
curve_meta, 12
curve_model, 14
curve_overlap, 16
curve_overlap(), 32
curve_rev, 8, 17
curve_rev(), 8, 9, 19, 25
curve_snowflake, 18
curve_snowflake(), 20, 26
curve_snowflake_batch, 19
curve_snowflake_batch(), 19
curve_summary, 21
curve_surv, 22
curve_table, 4, 23
curve_table(), 6, 21, 25, 26
curve_wrap, 14, 15, 24
curve_wrap(), 15

export_for_powerbi, 25
export_for_powerbi(), 19

ggcurve, 4, 27
ggcurve(), 6, 15, 18, 23, 25, 30, 32

plot_compare, 29
plot_compare(), 6, 16, 18, 20, 23, 28, 32
plot_multi, 31

RobustMax, 33
RobustMin, 33